



1506
UNIVERSITÀ
DEGLI STUDI
DI URBINO
CARLO BO

UNIVERSITÀ DEGLI STUDI DI URBINO CARLO BO

Dipartimento di Scienze Pure e Applicate
Corso di Laurea Magistrale in Informatica e Innovazione Digitale

Tesi di Laurea

CREAZIONE DI UN GESTORE DI MESSAGGI PER LA SOMMINISTRAZIONE DI TEST IN ERLANG

Relatore:
Chiar.mo Prof. Claudio Antares Mezzina

Candidato:
Paride Dominici

Correlatore:
Chiar.mo Prof. Alessandro Aldini

Anno Accademico 2024-2025

Alla mia famiglia

Indice

1	Introduzione	1
1.1	Scopo del progetto	1
1.2	Introduzione al problema	1
1.3	Possibili soluzioni	2
1.3.1	Aumento delle risorse	2
1.3.2	Allocazione dinamica delle risorse	2
1.3.3	Un nuovo approccio	3
1.3.4	Configurazione	4
2	Analisi dei requisiti	5
2.1	Comunicazione	5
2.1.1	Esempi di messaggi che possono essere utilizzati	5
2.1.2	Requisiti tecnici	6
2.2	Differenze tra le due modalità	6
2.3	Diagrammi degli stati	7
3	Strutture di comunicazione	10
3.1	Struttura JSON	10
4	Implementazione della comunicazione	12
4.1	Comunicazione tra client e gestore dei messaggi	12
4.1.1	Lato client Javascript	12
4.2	Lato gestore dei messaggi	14
4.2.1	Gestione delle diverse tipologie di connessione	15
4.2.2	Memorizzazione dei messaggi	16
4.3	Comunicazione tra gestore dei messaggi e server PHP	18
4.3.1	Limitazione del numero di connessioni al server PHP	18
5	Verifica della funzionalità	21
6	Conclusioni	22
A	Appendice A	23

Bibliografia	24
Ringraziamenti	25

Elenco delle figure

1.1	Configurazione attuale	2
1.2	Nuova configurazione	3
2.1	Esempio comunicazione senza gestore messaggi	6
2.2	Esempio comunicazione con gestore messaggi	7
2.3	Diagramma degli stati del Client	8

Elenco delle tabelle

3.1	Elenco dei comandi che possono essere inviati dal client	10
3.2	Elenco dei comandi di risposta da parte dei server	11
4.1	Struttura dei record della tabella mnesia	17

Capitolo 1

Introduzione

1.1 Scopo del progetto

Il progetto si propone l'obiettivo di verificare la possibilità di utilizzare le peculiarità del linguaggio erlang[3] o elixir[4] per migliorare la gestione dei trasferimenti dei dati nella somministrazione di test.

Nello specifico se sia possibile migliorare e rendere più robusto ed affidabile il trasferimento dei dati dalle postazioni degli studenti al server che si occupa di raccogliere ed elaborare i risultati.

I problemi principali che ci proponiamo di risolvere sono l'affidabilità nel trasferimento dei dati a causa di problemi di connettività e la garanzia di congruenza dei dati stessi.

Per fare questo si vuole creare una rete di nodi (da stabilire) che permettano alle postazioni studente di inviare sia le versioni temporanee che quelle definitive delle risposte al test, ai server che raccolgono i risultati e realizzare anche, sempre tramite la rete di nodi erlang una rete affidabile e resistente ai problemi.

1.2 Introduzione al problema

Al momento la somministrazione dei test al PC utilizzando i browser web utilizzano una architettura di tipo Client/Server per gestire la comunicazione tra il server che gestisce e distribuisce il test e i vari client che registrano ed inviano le risposte inserite dagli studenti.

In linea di massima lo schema è quello visibile in figura 1.1.

In questa modalità ogni client fa una serie di chiamate al server web che genera dinamicamente la pagina da visualizzare sul browser del client. Questa pagina viene generata diversamente in base allo stato in cui si trova il client. Ad esempio potrebbe essere la pagina per fare il login alla piattaforma oppure quella in cui si inseriscono i propri dati passando per le pagine in cui ci sono le domande del test con i campi in cui l'utente inserisce la risposta esatta fino ad arrivare alla pagina con il punteggio realizzato del test.

Tutte queste interazioni avvengono, attraverso chiamate http o https e sfruttano direttamente le risorse della macchina su cui è installato il web server.

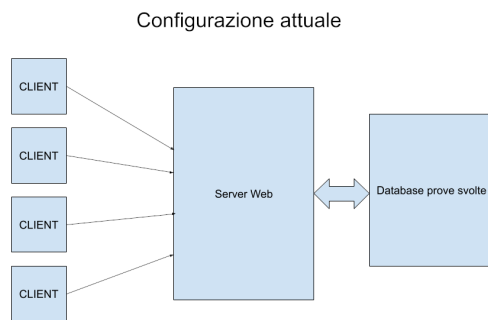


Figura 1.1: Configurazione attuale

All'aumentare del numero di utenti che si collegano aumenta anche la quantità di risorse che vengono utilizzate fino ad arrivare ad un punto, dipendente dall'architettura e dalle risorse presenti sulla macchina che impedisce al web server di funzionare correttamente provocando una serie di errori.

1.3 Possibili soluzioni

1.3.1 Aumento delle risorse

La prima soluzione è quella più comune è quella di aumentare le risorse disponibili per il web server.

Questo si può fare acquistando nuovo hardware e installando sulla nuova macchina un nuovo web server che però va configurato ed ottimizzato correttamente e richiede un investimento periodico, oltre che economico per l'acquisto dell'hardware, anche di tempo per l'installazione, la configurazione e il trasferimento dei dati tra una macchina e l'altra.

1.3.2 Allocazione dinamica delle risorse

Un'altra soluzione può essere quella di installare il nostro web server, fin dal primo momento, in un sistema che permetta una scalabilità delle risorse dinamica senza essere legati direttamente all'hardware presente.

Un primo livello è creare una server farm interna all'azienda che permetta la gestione delle macchine in ambiente virtuale. In questo caso lo sfruttamento delle risorse viene gestito e diviso, tra le risorse disponibili, da una infrastruttura di base che nasconde il vero hardware.

Un secondo livello è quello di affidarsi a sistemi distribuiti a livello geografico come Amazon AWS, Microsoft Azure, ecc. che permettono anche di allocare dinamicamente le risorse in base al carico di lavoro che è in quel momento presente sul nostro sistema.

In questi casi l'investimento dal punto di vista temporale, rispetto al caso precedente, viene quasi annullato dato che é sufficiente, per aumentare le risorse cambiare qualche parametro anche a macchine accese, per assegnare le risorse necessarie a sostenere il carico di lavoro che si presenta.

D'altro canto l'investimento economico in questi ultimi due casi é necessariamente piú grande di quello richiesto dal primo caso di cui abbiamo parlato.

1.3.3 Un nuovo approccio

Il lavoro che verrà portato avanti in questa tesi si pone come obiettivo quello di trovare un sistema per evitare il problema del sovraccarico del server web da parte di troppe connessioni contemporanee senza dover investire in nuove risorse hardware.

Per far questo creeremo in layer intermedio tra il web server e i client come in Figura 1.2.

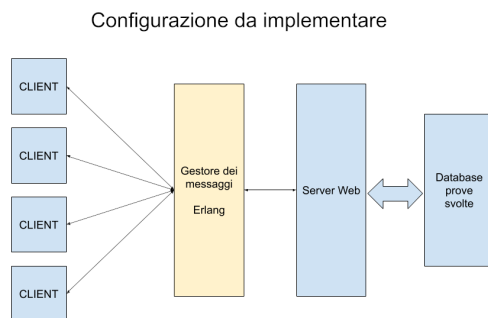


Figura 1.2: Nuova configurazione

Questo nuovo layer deve occuparsi, usando dei messaggi, di fare da tramite tra i client e il server web gestendo le richieste in modo che non ne arrivino troppe contemporaneamente al web server.

Sfortunatamente questo richiederá anche di cambiare il modo in cui i client e il server comunicano tra di loro.

In particolare non sarà piú direttamente il server a generare la pagina da visualizzare sul client ma passeremo ad una gestione piú dinamica della creazione delle pagine direttamente sul client in javascript con un approccio di tipo Client Side Rendering o meglio ancora DOM Manipulation.

Questo approccio richiede che il client abbia un browser piú moderno dato che una parte del processo viene spostata lato client. D'altro canto ormai tantissime applicazioni, come ad esempio Google mail o Facebook, hanno questo tipo di approccio.

Gli scambi tra il client, il gestore di messaggi e il server PHP, avverranno quindi solo tramite messaggi codificati in JSON e non come pagine web intere. Questo ci permetterà,

in ottica futura, di poter modificare una singola parte (client, server o gestore di messaggi) senza dover modificare obbligatoriamente le altre.

1.3.4 Configurazione

Per verificare il nostro progetto realizzeremo un server di test nel quale installeremo il server Yaws nel quale girerà il codice erlang ed anche una configurazione standard di apache, php, mysql (o mariadb) in modo da poter provare l'efficacia del nostro sistema.

Implementeremo comunque, sia lato client, che lato server php, un set minimo di comandi sufficiente per provare la risposta del server php interrogato direttamente dal client html confrontato con la stessa configurazione ma con il gestore di messaggi inserito tra i due elementi.

Capitolo 2

Analisi dei requisiti

2.1 Comunicazione

La prima cosa che dobbiamo analizzare é la tipologia di messaggi che possono essere scambiati tra il gestore dei messaggi, i client e il server che gestisce i test.

Per avere un riferimento stiliamo innanzitutto un elenco dei messaggi che potrebbero essere scambiati tra il server e i client in una situazione standard.

2.1.1 Esempi di messaggi che possono essere utilizzati

- Registrazione nuova postazione
- De-registrazione postazione
- Richiesta codice di accesso
- Invio codice di accesso
- Richiesta invio dati studente
- Invio dati studente
- Richiesta test da somministrare
- Invio contenuto del test da somministrare
- Invio risposte test
- Termine forzato del test
- Mostra messaggio sulla postazione
- Richiesta stato postazione

2.1.2 Requisiti tecnici

Per poter comunicare sia lato client che lato server, visto che le comunicazioni avvengono già su http o https non useremo erlang direttamente ma ci affideremo al web server Yaws [5] che è stato implementato proprio in Erlang e che permette di inserire all'interno del codice html, in modo simile a php, direttamente chiamate a codice erlang. Saremo costretti ad adattare il codice del client e del server in modo che comunichino attraverso il nuovo gestore dei messaggi.

2.2 Differenze tra le due modalità

La differenza principale, nella comunicazione tra client e server è che nella configurazione originale, senza il gestore dei messaggi, ogni richiesta del client viene gestita direttamente dal server che risponde, nella stessa connessione, con la pagina o il messaggio relativo.

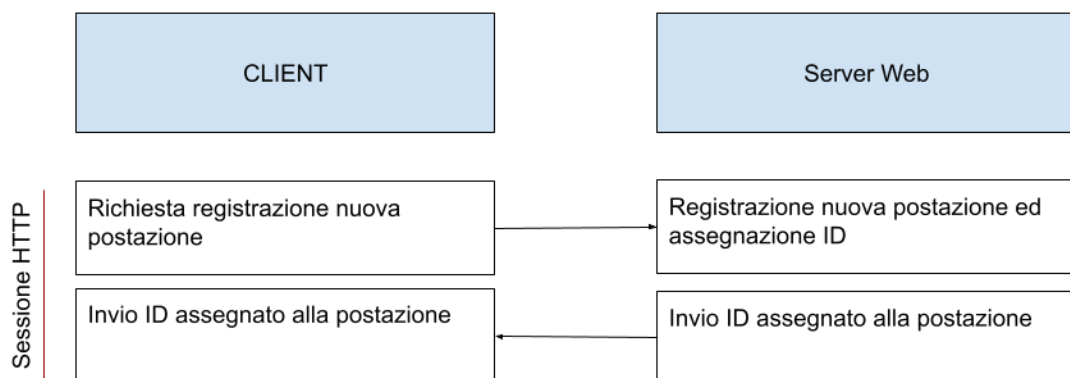


Figura 2.1: Esempio comunicazione senza gestore messaggi

Nella configurazione con il gestore dei messaggi invece la comunicazione avviene in modo asincrono. Il client fa una richiesta al gestore dei messaggi, questo risponde nella stessa comunicazione con l'ID con il quale la richiesta è stata memorizzata nella coda dei messaggi in modo che il client possa sapere quando la richiesta è stata processata.

Ad intervalli regolari il client farà una richiesta al gestore dei messaggi per sapere l'esito della sua richiesta.

La comunicazione tra gestore dei messaggi e server php la comunicazione avverrà in modo simile. Il gestore dei messaggi prenderà uno dei messaggi dalla coda e si collegherà al server php. Per evitare le congestioni implementeremo un sistema che limiti il numero di processi contemporanei attivi. Se tutte le connessioni che abbiamo deciso fossero già occupate il gestore di messaggi metterà il messaggio di nuovo nella coda ed attenderà un tempo variabile prima di tentare nuovamente.

Se la connessione va a buon fine il server php ci risponderà con il risultato della richiesta. Questa verrà nuovamente memorizzata nella coda dei messaggi di risposta per essere inviata al client corrispondente all'ID della richiesta originale.

Nel caso della comunicazione con il server avremo una comunicazione sincrona anziché asincrona.

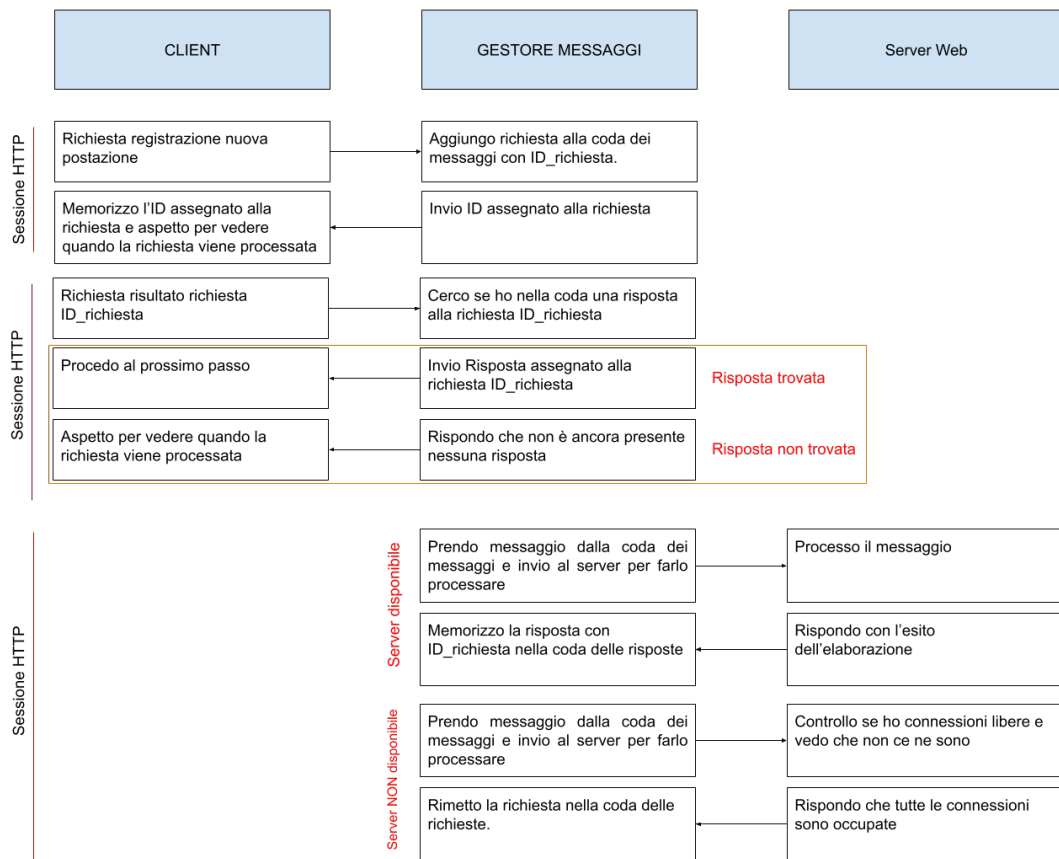


Figura 2.2: Esempio comunicazione con gestore messaggi

2.3 Diagrammi degli stati

Per comprendere meglio quali sono i messaggi che saranno necessari per la comunicazione con il server useremo dei diagrammi degli stati.

Sul client, per l'esecuzione di un test la sequenza delle operazioni che questo dovrà eseguire è la seguente:

1. Registrazione postazione sul server PHP*
2. Inserimento del codice della prova da svolgere nella pagina web da parte dell'utente

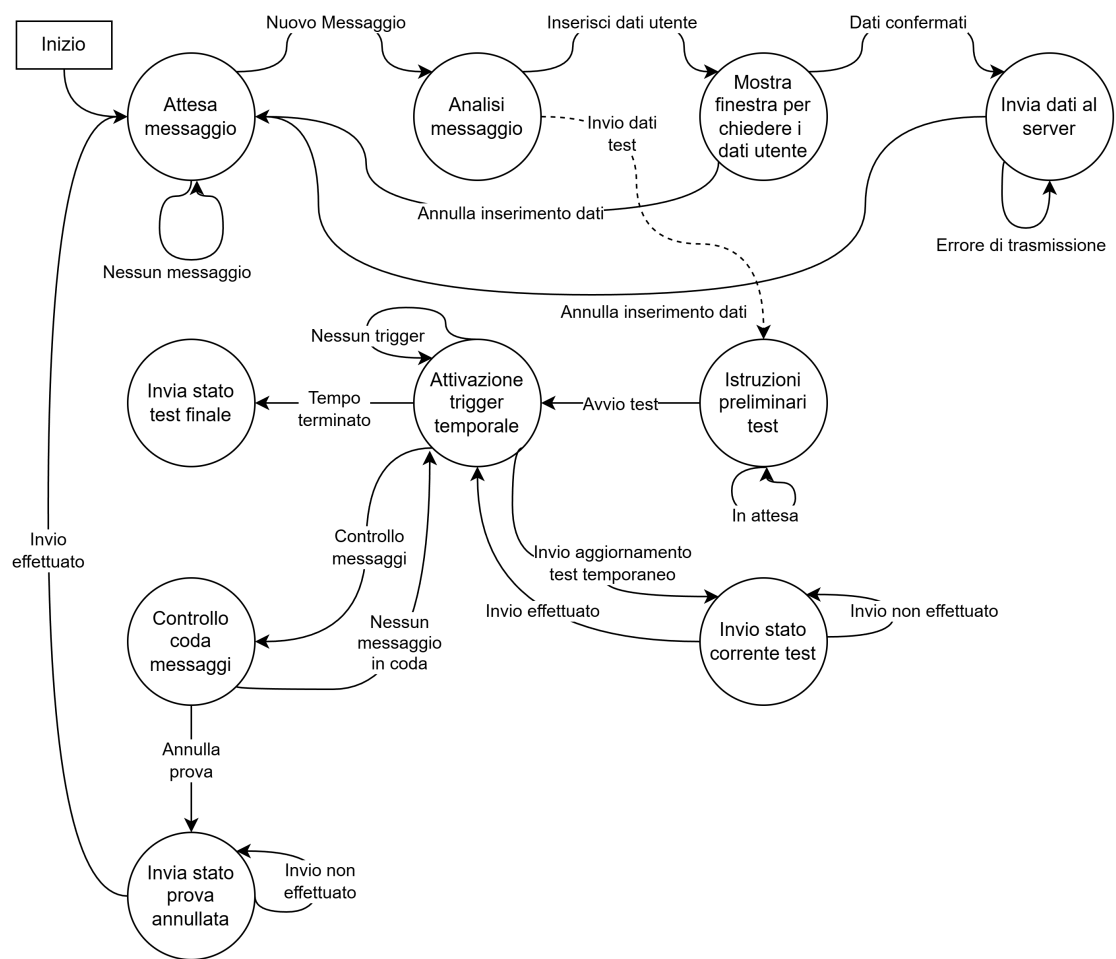


Figura 2.3: Diagramma degli stati del Client

3. Invio richiesta dati del test in base al codice inserito*

- Se il codice non é corretto scrivo Codice non corretto e lo faccio inserire di nuovo
- Se il codice é corretto usa i dati ricevuti dal server per creare dinamicamente la pagina web in base alle caratteristiche del test

4. Avvio i timer per lo svolgimento del test

- Invio dello stato temporaneo del test (ogni 30s)*
- Controllo se ci sono messaggi o azioni richieste dal server (interruzione test, messaggio a video, ecc.)*

5. Consegna test definivo*

- Consegna manuale prima della scadenza
- Consegna per scadenza del tempo

6. Cancellazione registrazione della postazione*

* vuol dire che c'é una richiesta HTTP

Capitolo 3

Strutture di comunicazione

La comunicazione tra il client, il gestore dei messaggi e il server avvengono attraverso strutture json.

3.1 Struttura JSON

La struttura JSON che viene inviata per la comunicazione tra i vari elementi dovrà avere uno schema del tipo:

```
{
  msg_type: tipo_messaggio,
  msg_content: contenuto_del_messaggio
}
```

I messaggi di ritorno avranno una struttura del tipo:

```
{
  result_type: tipo_messaggio_di_ritorno,
  result_content: contenuto_del_messaggio
}
```

Codifichiamo quindi i messaggi per il gestore dei messaggi:

Tabella 3.1: Elenco dei comandi che possono essere inviati dal client

test_message	Serve per fare un semplice test di comunicazione
register_station	La postazione vuole essere registrata sul sistema
unregister_station	La postazione vuole essere tolta dal sistema
request_code	La postazione vuole ricevere il test associato al codice inviato
send_test_content_backup	Invia al sistema una versione temporanea del test
send_test_content	Invia al sistema la versione definitiva del test

Il contenuto del messaggio dipenderà dal tipo di messaggio. Ad esempio se il tipo di messaggio é *test_message* il server si aspetterà anche il parametro *numero_iterazioni*.

Allo stesso modo dobbiamo codificare i messaggi di risposta dei server nel caso in cui l'esecuzione vada a buon fine:

Tabella 3.2: Elenco dei comandi di risposta da parte dei server

Messaggio in arrivo	Messaggio di risposta
test_message	test_result
register_station	registration_result
unregister_station	unregistration_result
request_code	test_content
send_test_content_backup	send_test_result
send_test_content	send_test_result

Nel caso in cui usiamo il gestore di messaggi, la comunicazione sarà asincrona per cui, una volta inviato il messaggio da parte del client, nella risposta ci sarà l'id della richiesta e, ad intervalli regolari, il client potrà verificare se la richiesta é andata a buon fine come indicato nella figura 2.2.

Capitolo 4

Implementazione della comunicazione

Per l'implementazione ci affideremo al web server implementato in Erlang[3], Yaws[5].

Yaws usa un sistema simile a quello di PHP. Permette infatti di includere codice erlang direttamente all'interno della pagina WEB inserendolo all'interno delle coppia di tag `<erl>` e `</erl>`.

La comunicazione tra client e server avviene tramite il passaggio di messaggi attraverso il gestore dei messaggi in erlang.

Questa si compone di due fasi distinte: la comunicazione tra il client e il gestore di messaggi e la comunicazione tra il gestore di messaggi e il server vero e proprio.

Nella prima il client comunicherà con il gestore dei messaggi attraverso semplici connessioni http o https in get o post che verranno processate dall'interprete erlang in Yaws.

Nella seconda fase il gestore di messaggi, comunicherà con web server PHP. Per evitare che si ripresenti lo stesso problema che avevamo con troppi client implementeremo una procedura che permetta solo un numero massimo di connessioni contemporanee. Sarà il gestore dei messaggi a permettere a tutti i client di collegarsi contemporaneamente e ad inviare i messaggi ricevuti al server al ritmo che il server accetta.

4.1 Comunicazione tra client e gestore dei messaggi

4.1.1 Lato client Javascript

Visto che il client gira su un browser la comunicazioni verrà gestita tramite codice Javascript. La comunicazione avverrà usando REST.

Utilizziamo per questo come base per la comunicazione alcune funzioni javascript:

```
let cache = {};  
const request = (url, params = {}, method = "GET", customHeaders = {}) => {  
  let cacheKey = JSON.stringify({ url, params, method });  
  if (cache[cacheKey]) {  
    return cache[cacheKey];  
  }  
}
```

```
// Definiamo le opzioni base inclusi gli headers
let options = {
  method,
  headers: {
    "Content-Type": "application/json", // Default per API moderne
    ...customHeaders
  }
}; if ("GET" === method) {
  url += "?" + new URLSearchParams(params).toString();
} else {
  options.body = JSON.stringify(params);
}

const result = fetch(url, options).then((response) => response.json());
cache[cacheKey] = result;

return result;
};
const get = (url, params, headers) => request(url, params, "GET", headers);
const post = (url, params, headers) => request(url, params, "POST", headers);
```

per passare i parametri al gestore dei messaggi metteremo i valori in un array

```
const dati =
{
  "msg_type": "test_messaggio",
  "contenuto": "Contenuto messaggio"
};
```

e poi li convertiranno in json

```
const json = JSON.stringify(dati);
```

infine invocheremo la funzione per inviare i dati e leggere il risultato della elaborazione

```
post("server.yaws", json, headers).then((post_res) => {
  const result_type = post_res.risposta.result_type;
  const risposta = post_res.risposta.result;
});
```

Il contenuto di risultato dipenderà logicamente dal comando che avremo inviato pur avendo la struttura di base che abbiamo definito nel capitolo 3.

4.2 Lato gestore dei messaggi

Dal lato del gestore dei messaggi dovremo gestire le richieste fatte in json ed analizzare i dati che ci verranno inviati. Nel server Yaws la funzione che viene eseguita per prima é la funzione `out()`.

```
out(Arg) ->
  Method = method(Arg),
  Risultato = try
    processa_comandi(Method, Arg)
  catch
    _ -> json:encode({
      <<"title">> => <<"Arrivato messaggio in POST">>,
      <<"method">> => <<"POST">>
    })
  end,

  %% Definiamo l'header CORS
  % Possiamo sostituire "*" con "https://tuosito.com" per maggiore sicurezza
  ACAO = {header, {"Access-Control-Allow-Origin", "*"}},

  % Torno gli header per evitare ACAO
  [
    ACAO,
    {content, "application/json", Risultato}
  ]
  .
```

Alla funzione vengono passati una serie di parametri inviati dal client. Questi parametri sono passati dentro alla variabile `Arg` che é un record e contiene diversi dati sulla connessione e sulla richiesta inviata al server web. Tra i piú importanti troviamo:

Dati della Richiesta

- **Arg#arg.clidata**: Contiene il corpo (body) della richiesta (es. dati POST o JSON).
- **Arg#arg.req**: Contiene un altro record con i dettagli della riga di richiesta HTTP (metodo, percorso, versione).
 - **Arg#arg.req#http_request.method**: L'atomo del metodo (es. 'GET', 'POST').
 - **Arg#arg.req#http_request.path**: Il percorso URL (es. `abs_path, "/index.yaws"`).
- **Arg#arg.querydata**: Contiene la stringa dei parametri dopo il ? in un URL (es. in `test.yaws?id=1`, contiene `"id=1"`).

I parametri che a noi interessano principalmente sono il metodo di connessioni (GET o POST) perché i dati devono essere prelevati in modo differente, e le due variabili *clidata* per le chiamate in modalità POST e *querydata* per le chiamate in modalità GET.

Vedremo meglio come gestire questa differenza più avanti. Nel codice sopra possiamo notare una sezione aggiuntiva che restituisce degli header aggiuntivi rispetto alla richiesta. Questi header si rendono necessari perché i browser, per evitare che ci possano essere attacchi di tipo XSS (Cross Site Scripting) impediscono collegamenti javascript che siano diretti al di fuori del proprio collegamento.

Nella nostra macchina di test, per poter eseguire contemporaneamente i server http PHP e YAWS abbiamo dovuto differenziare le porte per la connessione. Il server http PHP è stato impostato per lavorare sulla porta 80, mentre il server http yaws è stato impostato per lavorare sulla porta 8080. Avendo messo il client sulla porta 80, per il browser qualsiasi collegamento, compresi quelli sulla stessa macchina ma su una porta diversa, vengono visti come collegamenti esterni.

Per ovviare a questo problema è necessario che il server conceda, tramite un parametro dell'header la possibilità di accesso da un sito esterno. Per far questo aggiungiamo alla risposta "Access-Control-Allow-Origin" mettendo come sito autorizzato a collegarsi "*" che significa che tutti i siti possono accedere a lui.

4.2.1 Gestione delle diverse tipologie di connessione

La funzione *processa_comandi* si occupa di differenziare la gestione dei messaggi in base alla tipologia di connessione. In questo momento ci concentreremo sui comandi in POST anche se implementeremo comunque anche la gestione di quelli in GET per poter fare dei test di connessione.

Innanzitutto creeremo una mappa con i parametri da restituire indietro. I parametri principali saranno: *title*, *method*, *result*, risposta.

I primi due servono principalmente per verificare che la connessione funzioni, in *result* metteremo gli stessi parametri inviati dal client, anche questi per verificare il corretto funzionamento, ed infine in *risposta* ci sarà la struttura json con all'interno le informazioni che sono indirizzate al client.

Cercheremo di intercettare eventuali problemi con la gestione dei parametri tramite l'utilizzo delle clausole *try ... catch*.

La struttura dei dati verrà infine convertita in json per essere inviata indietro dal server tramite la funzione *json:encode()*. Utilizziamo la libreria *json* invece di quella proposta nel libro su Yaws (*rfc4627*) perché quest'ultima necessita di installazione ed è obsoleta. Fortunatamente, a partire dalla versione **Erlang/OTP 27** è stata integrata direttamente nel linguaggio la libreria json che essendo nativa è anche più veloce e ottimizzata.

```
processa_comandi('POST', Arg) ->
  Parametri = #{
    <<"title">> => <<"Arrivato messaggio in POST">>,
    <<"method">> => <<"POST">>,
    <<"result">> =>
```

```

        try
            get_data(Arg)
        catch
            _:_ -> <<"result_error">>
        end,
        <<"risposta">> =>
        try
            process_data(Arg)
        catch
            _:_ -> <<"process_data_error">>
        end
    },
    Result = json:encode(Parametri),
    Result;

processa_comandi('GET', _Arg) ->
    %equivalente con le mappe di quello fatto fino ad ora
    Parametri = #{
        <<"title">> => <<"Arrivato messaggio in GET">>,
        <<"method">> => <<"GET">>,
        <<"result">> => <<"test_get">>
    },
    Result = json:encode(Parametri),
    Result;

processa_comandi(Method, Arg) ->
    Result = io_lib:format("Metodo non gestito: ~p", [Method]),
    Result
.

```

4.2.2 Memorizzazione dei messaggi

Per memorizzare i messaggi in transito tra il client e il server, avvenendo la comunicazione in modo asincrono, ci avvaleremo del database nativo erlang mnesia[6] che ci permetterà di memorizzare in modo efficace i vari comandi da inviare al server PHP ed anche le risposte di questo da inviare al client javascript.

Dato che i messaggi inviati dal client e quelli resituiti dal server hanno una struttura simile e che i codici dei messaggi non sono sovrapponibili useremo una sola tabella per la coda dei messaggi.

I record della tabella avranno la seguente struttura:

Creiamo quindi un file che permetta di inizializzare il database che dovrà contenere la tabella mnesia.

```
out(Arg) ->
```

Tabella 4.1: Struttura dei record della tabella mnesia

Nome campo	Significato
id	Identificativo del messaggio
msg_type	Tipo di messaggio
payload	Contenuto del messaggio
stato	Stato del messaggio (non ancora usato)
timestamp	Data e ora di inserimento del dato nel DB (Serve per poter fare pulizia)

```

Risultato = try
  esegui_inizializzazione()
catch
  _:Errore -> #{
    <<"result">> => <<"error">>,
    <<"message">> => list_to_binary(io_lib:format("~p", [Errore]))
  }
end,
[
  {header, {"Access-Control-Allow-Origin", "*"}},
  {content, "application/json", json:encode(Risultato)}
].

esegui_inizializzazione() ->
  Passi = [
    {"schema", fun crea_schema/0},
    {"mnesia", fun avvia_mnesia/0},
    {"tabelle", fun crea_tabelle/0}
  ],
  Risultati = lists:map(fun({Nome, Funzione}) ->
    Esito = try
      Funzione(),
      <<"ok">>
    catch
      _:Motivo -> list_to_binary(io_lib:format("~p", [Motivo]))
    end,
    {list_to_binary(Nome), Esito}
  end, Passi),

  application:set_env(yaws, db_inizializzato, true),

  maps:from_list([
    {<<"result">>, <<"ok">>},
    {<<"passi">>, maps:from_list(Risultati)}
  ])

```

```

]).

crea_schema() ->
  case mnesia:create_schema([node()]) of
    ok -> ok;
    {error, {_, {already_exists, _}}} -> already_exists
  end.

avvia_mnesia() ->
  case mnesia:start() of
    ok -> ok;
    {error, {already_started, _}} -> already_started
  end.

crea_tabelle() ->
  case mnesia:create_table(richiesta, [
    {attributes, [id, msg_type, payload, stato, timestamp]},
    {type, set},
    %{disc_copies, [node()]}
    {ram_copies, [node()]}
  ]) of
    {atomic, ok} -> ok;
    {aborted, {already_exists, _}} -> already_exists
  end.

```

Nel codice erlang del server yaws mettiamo la gestione di scrittura e lettura dei vari messaggi:

```

<<"test_message_m">> ->
  % Salva la richiesta in Mnesia
  Id = inserisci_richiesta(Tipo_Messaggio, Msg_content),
  Parametri_res = #{
    <<"result_type">> => <<"test_result_m">>,
    <<"result_content">> => #{
      <<"result_desc">> => <<Prefisso/binary>>,
      <<"id_message">> => integer_to_binary(Id)
    }
  },
  Parametri_res;
<<"test_message_r">> ->
  % Leggi la richiesta in Mnesia
  Id = maps:get(<<"Msg_content">>, Parametri, <<"default_value">>),
  case leggi_richiesta(Id) of
    {ok, Result} ->

```

```

Parametri_res = #{
  <<"result_type">> => <<"test_result_r">>,
  <<"result_content">> => #{
    <<"result_desc">> => <<"reading_result">>,
    <<"content">> => integer_to_binary(Id)
  }
};
{error, not_found} ->
Parametri_res = #{
  <<"result">> => <<"error">>,
  <<"message">> => <<"id non trovato">>
};
{error, Reason} ->
Parametri_res = #{
  <<"result">> => <<"error">>,
  <<"message">> => list_to_binary(io_lib:format("~p", [Reason]))
}
end,
Parametri_res;

```

4.3 Comunicazione tra gestore dei messaggi e server PHP

4.3.1 Limitazione del numero di connessioni al server PHP

Ci sono diversi metodi per limitare il numero connessioni che il server PHP accetta dal gestore dei messaggi. Alcuni dei più comuni sono:

- Webhooks semplice (Push asincrono via HTTP)
- Semaphore (Semafori a livello di Sistema Operativo)
- Gestione tramite il Web Server (Nginx / PHP-FPM)
- Usare una tabella del database per gestire il numero di processi

Ognuno di questi ha dei vantaggi e degli svantaggi. Webhooks ad esempio ha il vantaggio che è il gestore dei messaggi ad inviare i messaggi ma rimane con lo stesso problema che abbiamo con la configurazione originale client/server; rischiamo infatti che il server di sovraccarichi. Semaphore d'altro canto richiede, per l'utilizzo dei semafori del sistema operativo, che il nostro web server si trovi su un distribuzione Linux. La gestione del numero massimo di processi modificando la configurazione del web server richiede invece la possibilità di modificare i file di configurazione, cosa che non sempre è possibile e che dipende dalla scelta della macchina su cui far girare i nostri sorgenti php. Potrebbe essere una configurazione condivisa senza possibilità di cambiare i parametri di base.

L'utilizzo di una tabella del nostro database mysql o mariadb per gestire il numero massimo di richieste in arrivo dal gestore dei messaggi sembra quindi essere la soluzione

migliore. Non siamo legati al sistema operativo e non abbiamo bisogno di accedere alla configurazione di PHP. Il database é già installato per memorizzare i risultati delle prove svolte.

Implementazione del codice per limitare le connessioni

Come già detto, per limitare le connessioni al server web PHP, implementeremo una gestione apposita.

Nel momento arriva una nuova connessione andremo a vedere, nella tabella delle connessioni attive, quante connessioni sono già in essere. Se il numero supera il numero massimo stabilito risponderemo alla chiamata con un messaggio che indichi il sovraccarico. In caso contrario aggiungeremo alla tabella un elemento ed eseguiremo il comando che ci é stato richiesto. Al termine del comando cancelleremo dalla tabella la riga corrispondente.

Potrebbe però capitare che, per errori nel codice php o altro, l'esecuzione si interrompa in modo inaspettato lasciando nella tabella una riga non più valida. L'accumulo di queste righe impedirà la ricezione di nuovi messaggi da parte del gestore dei messaggi. Implementeremo quindi una procedura di emergenza che, ad intervalli casuali di tempo, cancelli gli elementi della tabella più vecchi. Possiamo stabilire, come tempo massimo di permanenza nella tabella, un minuto. Sarà infatti molto improbabile che ci siano processi che richiedano più di una trentina di secondi per essere eseguiti.

Creiamo quindi la tabella nel database:

```
CREATE TABLE processi_attivi (  
id INT AUTO_INCREMENT PRIMARY KEY,  
token_univoco VARCHAR(64), -- Per sicurezza extra nella cancellazione  
data_inizio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
ultimo_heartbeat TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Creiamo quindi il codice per controllare se ci sono ancora slot disponibili:

```
function GetMaxSlots() {  
    return 10;  
}  
  
function SlotIsFree($db) {  
    // Pulizia automatica (probabilita' 5% per ogni chiamata)  
    if (rand(1, 100) <= 5) {  
        // Elimina i record piu' vecchi di 1 minuto (processi probabilmente crashati)  
        $db->query("DELETE FROM processi_attivi  
        WHERE data_inizio < NOW() - INTERVAL 1 MINUTE");  
    }  
    $max_slots = GetMaxSlots();  
    $count = $db->query("SELECT COUNT(*) FROM processi_attivi")->fetchColumn();
```

```
    if ($count > $max_slot) {  
        return false;  
    }  
    return true;  
}
```

Visto che é il codice che viene eseguito piú spesso inseriamo qui il codice che, in modo casuale, cancella i record di processi attivi che si sono bloccati.

Aggiungiamo quindi il codice per prenotare e sbloccare uno slot di esecuzione:

```
function LockSlot($db) {  
    $token = bin2hex(random_bytes(16)); // Generiamo un ID unico per questa esecuzione  
  
    // 1. Inseriamo il record per tentare di occupare uno slot  
    $stmt = $db->prepare("INSERT INTO processi_attivi (token_univoco) VALUES (?)");  
    $stmt->execute([$token]);  
    $mio_id = $db->lastInsertId();  
  
    return $mio_id;  
}  
  
function UnLockSlot($db, $mio_id) {  
    if (isset($mio_id)) {  
        $db->prepare("DELETE FROM processi_attivi WHERE id = ?")->execute([$mio_id]);  
    }  
}
```

Capitolo 5

Verifica della funzionalità

Capitolo 6

Conclusioni

Appendice A

Appendice A

Bibliografia

- [1] Francesco Cesarini e Simon Thompson, *Erlang Programming*, United States of America: O'Reilly, 2009
- [2] Zachary Kessin, *Programming Web Services with Erlang: Working with Yaws*, Great Britain: O'Reilly, 2012
- [3] *Index - Erlang/OTP* [Online] (26/11/2025). [Ultimo Accesso: 26/11/2025]. Disponibile: <https://www.erlang.org/>
- [4] *The Elixir programming language* [Online] (28/12/2025). [Ultimo Accesso: 28/12/2025]. Disponibile: <https://elixir-lang.org/>
- [5] *Yaws (web server) - Wikipedia* [Online] (28/12/2025). [Ultimo Accesso: 28/12/2025]. Disponibile: [https://en.wikipedia.org/wiki/Yaws_\(web_server\)](https://en.wikipedia.org/wiki/Yaws_(web_server))
- [6] *mnesia - A distributed key-value DBMS* [Online] (07/03/2026). [Ultimo Accesso: 07/03/2026]. Disponibile: <https://www.erlang.org/doc/apps/mnesia/mnesia.html>

Ringraziamenti

Vorrei ringraziare ...